

What is a PCB?

A **Process Control Block (PCB)** is a **data structure maintained by the Operating System** that stores all the information required to manage and execute a process.

Think of it as the **identity card or file** of a process.

Company Analogy

Imagine every employee in a company has an HR file.

The HR file contains:

- Employee ID
- Current department
- Current project
- Salary
- Leave balance
- Contact details

Whenever HR needs information about an employee, they don't ask the employee—they look at the file.

Similarly, the OS doesn't ask the process. It looks at the PCB.

Why do we need a PCB?

Suppose you have these running:

```
Chrome
VS Code
Spotify
Terminal
```

The CPU can execute only one process at a time (on a single core).

So the OS keeps switching:

```
Chrome
↓
VS Code
↓
Spotify
↓
Chrome
```

Whenever the OS pauses one process, it must remember:

- Which instruction it was executing
- What values were in the CPU registers
- Which files were open
- How much memory it owns

Where is all this stored?

 **In the PCB.**

Without a PCB, the OS wouldn't know how to resume the process from where it left off.

Contents of a PCB

A PCB typically contains:

+-----+	
Process ID (PID)	
Process State	
Program Counter	
CPU Registers	
CPU Scheduling Info	
Memory Management Info	
I/O Status Information	
Accounting Information	
+-----+	

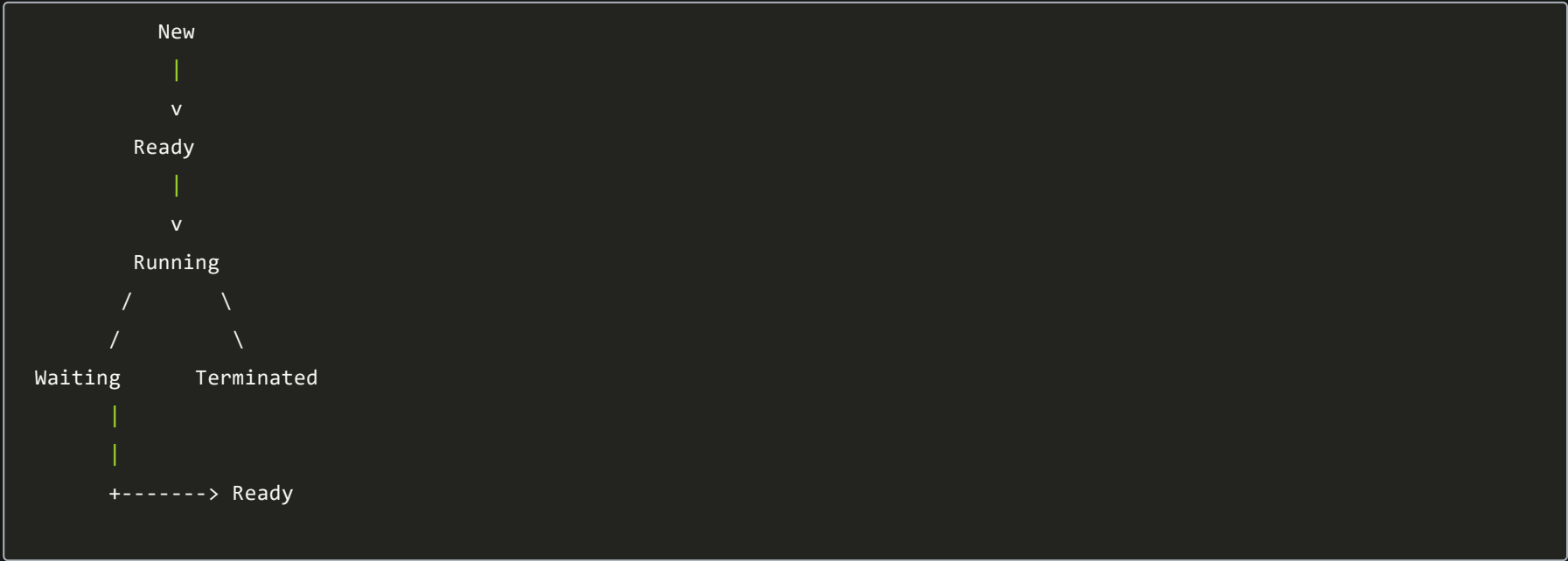
These are two closely related concepts:

- **Process States** → The different conditions a process can be in.
- **Process Lifecycle** → How a process moves from one state to another.

Process States

A process does not run all the time. It moves through different states during its lifetime.

The classic **5-state model** is:



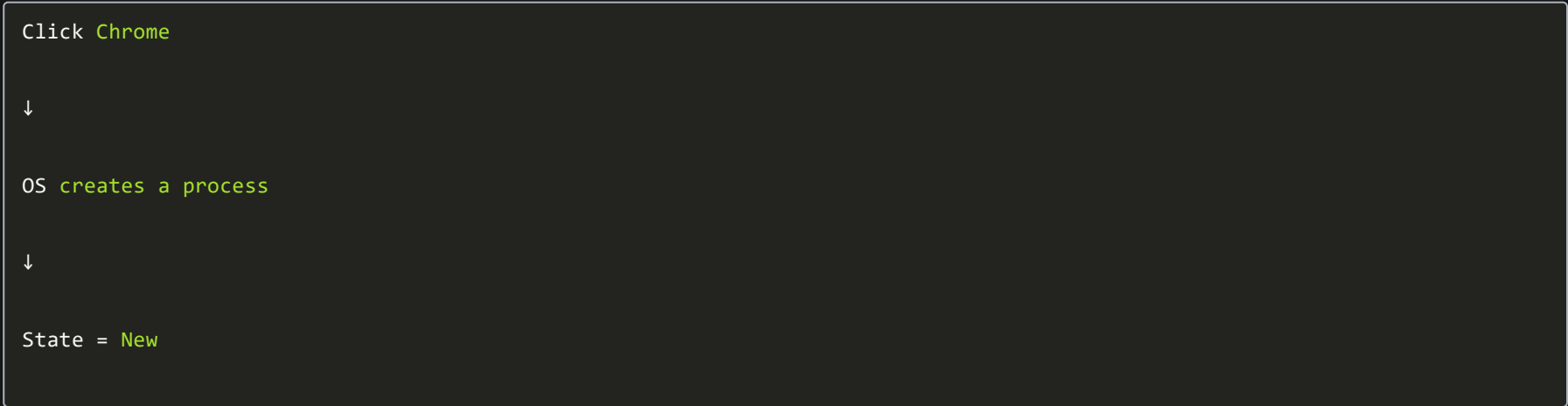
Let's understand each state.

1. New State

A process has just been created.

Example:

You double-click Chrome.



At this point, the OS is:

- Creating the PCB
- Allocating memory
- Assigning a Process ID (PID)
- Loading the program into memory

The process is **not yet ready to execute**.

2. Ready State

The process has everything it needs except the CPU.

Think of it like standing in a queue.

Example:

```
Chrome  
  
VS Code  
  
Spotify
```

All three are ready to run, but the CPU can only execute one process at a time (on a single-core CPU).

So they wait in the **Ready Queue**.

```
Ready Queue  
  
Chrome  
  
↓  
  
VS Code  
  
↓  
  
Spotify
```

The scheduler chooses one process from this queue.

3. Running State

The CPU is currently executing the process.

Example:

```
int x = 5;  
x++;  
printf("%d", x);
```

The CPU is executing these instructions.

Only one process per CPU core can be in the Running state at a time.

4. Waiting (Blocked) State

Sometimes a process cannot continue because it is waiting for an event.

Examples:

- Reading a file
- Waiting for keyboard input
- Waiting for network data
- Waiting for disk I/O

Example:

```
cin >> name;
```

Until the user types something, the process waits.

Or:

```
read(file);
```

The process waits until the disk returns the data.

While waiting, the CPU is free to execute another process.

5. Terminated State

The process has finished execution or has been killed.

Example:

```
return 0;
```

or

```
User closes Chrome.
```

The OS then:

- Frees memory
- Closes files
- Removes the PCB
- Releases resources

The process no longer exists.

FIVE-STATE MODEL

A process goes through five different states during its lifetime.

